ALEXANDER ARRICO

Senior Software Engineer
Brunswick, GA · alex@arrico.me · (310) 740-5037
arrico.me · linkedin.com/in/aarrico · github.com/aarrico

PROFESSIONAL SUMMARY

Senior software engineer with 10+ years modernizing production backends that have to stay online while they change. Led a Java-to-Node.js migration of 20,000+ users without data loss or full-service outages, and designed AWS messaging that handles 350,000+ outbound messages daily. Former engineering manager who remains hands-on with architecture, code, and production operations across Python, TypeScript, Java, and AWS.

PROFESSIONAL EXPERIENCE

Software Consultant | Freelance (Remote) November 2023 – May 2026

Sole engineer on a client's Discord community platform, owning architecture, deployment, and production operations end to end.

- Rewrote the client's TypeScript/Node.js application with a layered domain architecture and PostgreSQL persistence through Prisma.
- Built scheduled session automation with downtime-recovery logic, plus an auto-discovery pattern for command and event handlers that let new features register with zero configuration.
- Replaced manual deployments with multi-stage Docker builds and GitHub Actions CI/CD on Railway. Containerized PostgreSQL kept local and production at parity.

Engineering Manager | Hourwork, Inc. (Boston, MA) May 2022 – November 2023

Player-coach on a 5-person cross-functional team (3 engineers, QA, product); primary backend engineer on the platform rebuild while managing the team.

- Architected a serverless messaging platform on AWS Lambda, SQS, and SNS with Twilio that carried 350,000+ outbound and 12,000 inbound messages daily.
- Re-architected the PostgreSQL data models and cut query latency from seconds to tens of milliseconds. The resulting real-time analytics expanded the enterprise sales pipeline.
- Migrated 20,000+ users from a Java/Spring Boot monolith to Node.js microservices in 11 months through staged customer rollouts with verification checks. Kept the old system running for unmigrated customers and paused rollouts to resolve scaling issues before expanding, with zero customer data loss and no full-service outages.
- Built out observability on DataDog and cut resolution time for critical production issues from weeks, or never, to under 24 hours.
- Moved releases from sporadic multi-month cycles to weekly with trunk-based development and automated CI/CD.
- Instituted code review standards, an RFC process, and incident management protocols; customer-reported bugs dropped 85% within two months.

Lead Software Engineer | BlueMark (Valencia, CA) June 2021 – April 2022

Backend lead on an e-commerce platform build, owning REST API design and the automated test strategy.

- Designed and deployed a REST API in Python/Flask on AWS EC2 with PostgreSQL, specifying the OpenAPI/Swagger contracts ahead of implementation so the frontend and backend teams could build in parallel instead of blocking on each other.
- Cut defects reaching QA and staging 70% with unit and integration coverage in pytest and Jest.
- Moved Postgres access onto SQLAlchemy, replacing hand-written queries with a single data-access layer.

Director of Technology | Optricity Corp. (Durham, NC) January 2019 – June 2021

Promoted to build the company's first engineering function while remaining the senior hands-on engineer on the flagship product.

- Re-architected the flagship product's XML-based project loading engine and brought load times for the largest projects from over 10 minutes to under two.
- Integrated a Python scripting engine into the core Java desktop application so customers could automate their own workflows, then used the same engine to add a regression-testing stage to CI. Customer-reported defects fell 60%.
- Built and shipped a serverless client portal on GCP (App Engine, Firebase) in under seven days, which kept client training and key business functions running through the COVID-19 shutdowns.
- Migrated issue tracking to GitLab and built automated CI/CD workflows.

Senior Software Engineer | Optricity Corp. (Durham, NC) January 2016 – January 2019

Founding engineer on the product team, promoted to Senior, building client-facing APIs and internal tooling.

- Built a Web Services API that let clients connect their Warehouse Management Systems directly to the product and cut 10+ hours of manual data entry per client each week.
- Developed a Python/Flask quality-control scanner that validated client CSV uploads before ingest. Clients got immediate feedback and support tickets dropped 20%.
- Architected the company's secondary support application in C#/.NET with proprietary algorithms for warehouse system integration.
- Led the flagship codebase migration from SVN to Git and built the company's first Jenkins CI/CD pipeline.

TECHNICAL SKILLS

Languages: Python, TypeScript, JavaScript, Node.js, Java, C#, SQL, Bash

Backend & APIs: FastAPI, Flask, Express.js, Next.js, REST API Design, OpenAPI/Swagger, Pydantic, asyncio

Data & Messaging: PostgreSQL, MongoDB, Elasticsearch, Redis, SQLAlchemy, Prisma, AWS SQS/SNS, Twilio, Event-Driven Architecture

Cloud & Infrastructure: AWS (Lambda, SQS, SNS, EC2, RDS, CloudWatch), GCP (App Engine, Firebase), Docker, Microservices, Serverless

DevOps & Testing: CI/CD (GitHub Actions, GitLab CI, Jenkins), DataDog, pytest, Jest, Cypress, Trunk-Based Development

Frontend: React, HTML/CSS

Practices: Incident Management, RFC/Architecture Review

PROJECTS

Starly | Distributed event processing platform github.com/aarrico/starly

Python, FastAPI, MongoDB, Elasticsearch, Redis, Docker

- Built a FastAPI event ingestion pipeline with an SQS-compatible in-process queue, async workers, MongoDB as source of truth, Elasticsearch full-text search, and Redis cache-aside for real-time stats.
- Designed dependency-specific failure handling: Redis outages recompute from MongoDB; storage or search outages return typed HTTP 503s. Under sustained overload, rejected 57% of writes with HTTP 503 and Retry-After when the bounded queue filled; requests accepted with HTTP 202 had no errors or data loss during the test.
- Measured worker drain from 50 to 251 events/sec across one to four concurrent consumers in single-node benchmarks. Traced the drain ceiling to contention on the API's shared event loop: drain ran 15x faster with ingest idle, while added consumers increased drain throughput at the cost of ingest p99 latency.
- Documented the queue's delivery guarantees, its divergences from real SQS, failure modes, and 10x scaling limits in an architecture RFC.

EDUCATION

M.S. Materials Science & Engineering
University of Tennessee, Knoxville 2012 – 2014

B.S. Physics / B.A. Computer Science / B.A. Mathematics
Duquesne University, Pittsburgh 2008 – 2012